



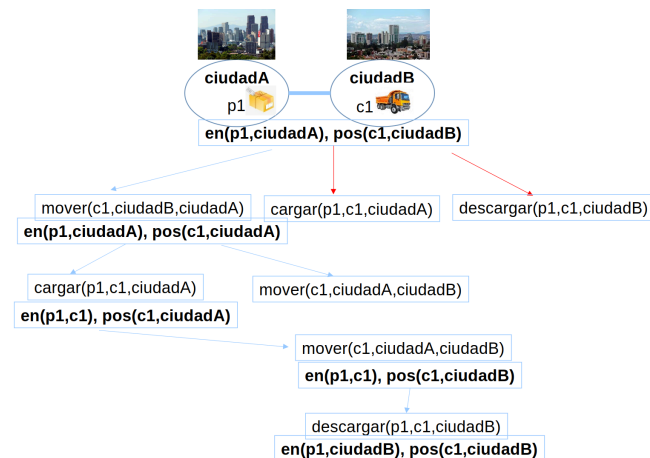
UNIVERSIDAD
SAN SEBASTIAN

USS-ICIFH001 / INTELIGENCIA ARTIFICIAL / CLASE 08

Clase 08 – STRIPS y PDDL

Cristhian Aguilera

2026-08-25



Contenidos

1. Objetivos de aprendizaje

2. Motivación

3. STRIPS

4. PDDL

5. Resumen

Objetivos de aprendizaje

Al finalizar esta clase, los estudiantes podrán:

Representar estados iniciales, metas y operadores mediante STRIPS.

Determinar si una acción es aplicable y calcular el estado que produce.

Distinguir la información reusable de un dominio PDDL y la de un problema particular.

Codificar y validar un dominio de transporte y una instancia resoluble.

01

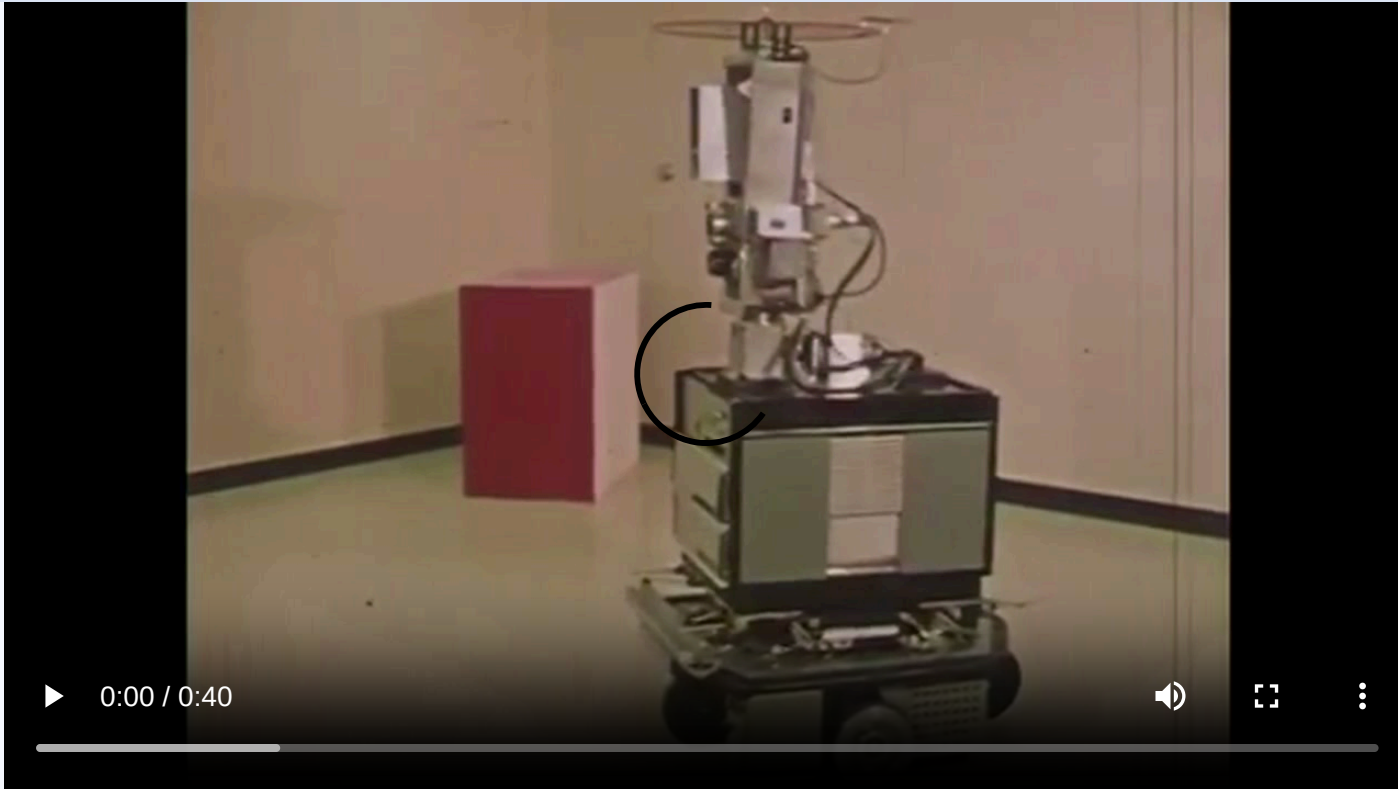
SECCIÓN

Motivación

1.1 Shakey

1.2 Del mundo al modelo

Shakey: hardware primitivo, software que cambió el mundo

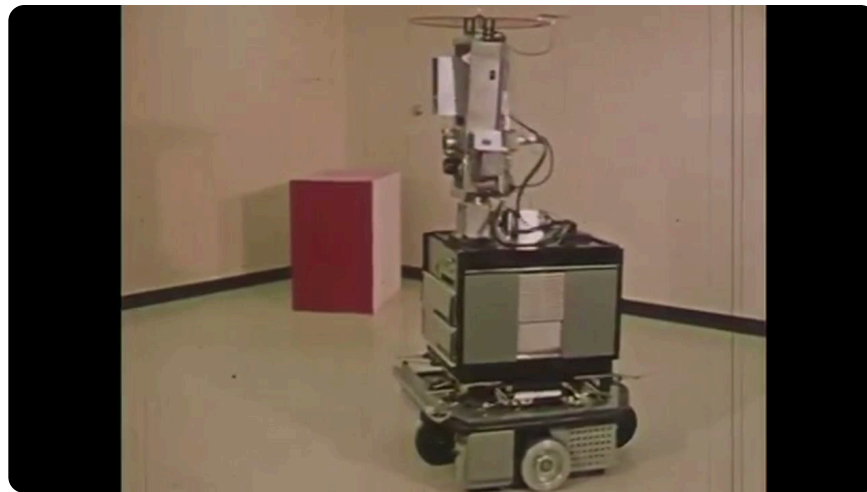


¿Qué tendría que saber su programa?

⌚ 5 min · Pares

Queremos darle una meta a Shakey, no programar cada movimiento.

- ¿Qué *hechos* debe conocer sobre el mundo antes de comenzar?
- Para cada acción, ¿cómo indicarían *cuándo se puede ejecutar* y *qué cambia* después?
- ¿Cómo escribirían esa descripción una sola vez para reutilizar las reglas con nuevas metas?



02

SECCIÓN

STRIPS

- 2.1 Estados iniciales y metas
- 2.2 Precondiciones y efectos
- 2.3 Planificación como búsqueda

Un problema STRIPS separa inicio, meta y operadores

$$P = \langle I, G, O \rangle$$

- I : estado inicial.
- G : estado meta.
- O : conjunto de operadores o acciones.

STRIPS —*Stanford Research Institute Problem Solver*— fue propuesto por Fikes y Nilsson en 1971. Representa estados y cambios mediante términos lógicos para que un algoritmo pueda buscar un plan.

Un estado es un conjunto de hechos verdaderos

Cada hecho se escribe como un *literal* o *predicado* aplicado a objetos:

pos(truck1, santiago)

at(package1, concepcion)

El conjunto de hechos describe las propiedades y relaciones relevantes del problema.

El estado inicial y la meta cumplen funciones distintas

Estado inicial I

$$\begin{aligned} &\{at(p1, city-a), \\ &\quad pos(t1, city-b), \\ &\quad road(city-a, city-b), \\ &\quad road(city-b, city-a)\} \end{aligned}$$

Estado meta G

$$\{at(p1, city-b)\}$$

La meta no tiene que describir el mundo completo: solo las condiciones que deben ser verdaderas al terminar.

Una acción declara cuándo se aplica y qué cambia

Cada acción contiene un *nombre*, *precondiciones* y *efectos*.

- *Add list*: hechos que pasan a ser verdaderos.
- *Delete list*: hechos que dejan de ser verdaderos.

Los hechos no mencionados por los efectos se mantienen sin cambios.

Mover cambia la posición del camión

La acción solo es aplicable cuando existe una ruta y el camión está en el origen.

Al ejecutarla, se agrega la nueva posición y se elimina la anterior.

```
1  move(truck, from, to)
2    pre:
3      road(from, to)
4      pos(truck, from)
5    add:
6      pos(truck, to)
7    delete:
8      pos(truck, from)
```

Cargar y descargar cambian la ubicación del paquete

`load(package, truck, city)`

- Pre: paquete y camión están en la ciudad.
- Add: el paquete queda en el camión.
- Delete: el paquete deja de estar en la ciudad.

`unload(package, truck, city)`

- Pre: el paquete está en el camión y este está en la ciudad.
- Add: el paquete queda en la ciudad.
- Delete: el paquete deja de estar en el camión.

Aplicar una acción produce un nuevo estado

Estado actual:

$$\{at(p1, city-a), pos(t1, city-b), road(city-b, city-a)\}$$

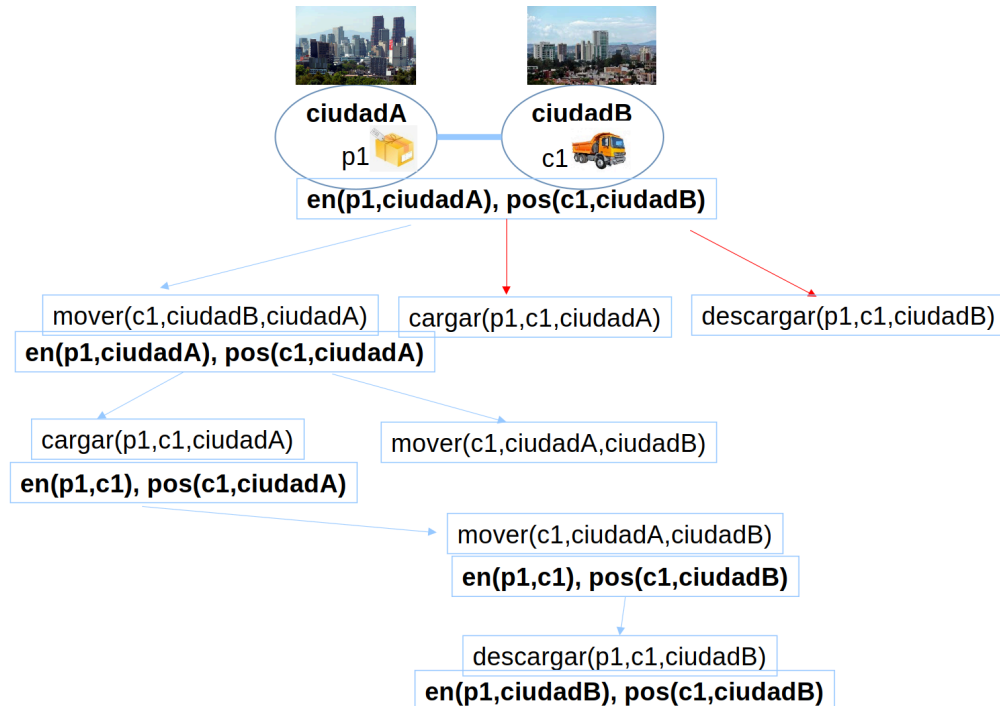
`load(p1,t1,city-a)` aún no es aplicable porque el camión no está en `city-a`.

`move(t1,city-b,city-a)` sí es aplicable:

- elimina $pos(t1, city-b)$;
- agrega $pos(t1, city-a)$;
- mantiene los demás hechos sin cambios.

En el estado resultante, `load(p1,t1,city-a)` pasa a ser aplicable.

BFS puede buscar sobre los estados definidos por STRIPS



Cada arista representa una acción aplicable y cada nodo, el estado resultante.

03

SECCIÓN

PDDL

3.1 Componentes de PDDL

3.2 Archivo de dominio

3.3 Archivo de problema

PDDL estandariza la descripción de problemas de planificación

PDDL —*Planning Domain Definition Language*— es el lenguaje estándar para representar problemas de planificación. Utiliza una sintaxis similar a LISP y puede ser procesado por distintos planificadores. Permite expresar objetos, predicados, estado inicial, meta y acciones sin depender de un planificador específico.

PDDL organiza los elementos del modelo simbólico

- *Objetos*: entidades concretas del problema.
- *Predicados*: propiedades o relaciones que pueden ser verdaderas o falsas.
- *Estado inicial y meta*: instancia que se quiere resolver.
- *Acciones*: precondiciones y efectos que transforman el estado.

Separar dominio y problema permite reutilizar las reglas

Archivo de dominio

- Nombre y requisitos del dominio
- Predicados disponibles
- Acciones, parámetros, precondiciones y efectos
- Reutilizable en muchas instancias

Archivo de problema

- Dominio que utiliza
- Objetos de esta instancia
- Estado inicial
- Condiciones de meta

Estructura de un archivo de dominio

Esta plantilla muestra dónde se declaran los predicados y operadores.

Los marcadores entre `<...>` se reemplazan al construir un dominio real.

```
1 (define (domain <domain-name>)
2   (:predicates <predicate-list>)
3   (:action <action-name>
4     :parameters (<parameter-list>)
5     :precondition <expression>
6     :effect <expression>
7   )
8 )
```

Estructura de un archivo de problema

Una instancia enlaza el dominio con objetos, hechos iniciales y una meta concreta.

```
1 (define (problem <problem-name>)
2   (:domain <domain-name>)
3   (:objects <object-list>)
4   (:init <initial-state>)
5   (:goal <goal-expression>)
6 )
```

El dominio de transporte declara sus predicados

- `at` : ubicación de un paquete.
- `pos` : posición de un camión.
- `road` : conexión dirigida entre ciudades.

El archivo completo es ejecutable; veremos sus fragmentos principales.

```
1  (:predicates
2    (at ?package ?place)
3    (pos ?truck ?city)
4    (road ?from ?to)
5  )
```

Mover: precondiciones y efectos

La forma PDDL corresponde directamente al operador STRIPS:

- la precondición exige ruta y posición inicial;
- el efecto agrega la nueva posición;
- `not` elimina la posición anterior.

```
1  (:action move
2    :parameters (?truck ?from ?to)
3    :precondition (and
4      (road ?from ?to)
5      (pos ?truck ?from))
6    :effect (and
7      (pos ?truck ?to)
8      (not (pos ?truck ?from)))
9  )
```

Cargar: el paquete pasa al camión

El paquete y el camión deben coincidir en una ciudad.

El efecto actualiza únicamente la ubicación del paquete.

```
1  (:action load
2    :parameters (?package ?truck ?city)
3    :precondition (and
4      (at ?package ?city)
5      (pos ?truck ?city))
6    :effect (and
7      (at ?package ?truck)
8      (not (at ?package ?city)))
9  )
```


Descargar: el paquete pasa a la ciudad

El paquete debe estar en el camión y el camión debe estar en la ciudad de descarga.

```
1  (:action unload
2    :parameters (?package ?truck ?city)
3    :precondition (and
4      (at ?package ?truck)
5      (pos ?truck ?city))
6    :effect (and
7      (at ?package ?city)
8      (not (at ?package ?truck)))
9  )
```

El problema declara los objetos de esta instancia

La instancia contiene dos paquetes, un camión y cuatro ciudades.

Estos objetos utilizarán los predicados y acciones definidos por `transport`.

```
1 (:objects
2   p1 p2
3   t1
4   city-a city-b city-c city-d
5 )
```

El estado inicial ubica objetos y conecta ciudades

El camión parte en `city-b`; los paquetes parten en `city-a` y `city-c`.

Las relaciones `road` determinan los movimientos aplicables.

```
1 (:init
2   (at p1 city-a)
3   (at p2 city-c)
4   (pos t1 city-b)
5   (road city-a city-b)
6   (road city-b city-a)
7   (road city-b city-c)
8   (road city-c city-b)
9   (road city-c city-d)
10  (road city-d city-c)
11 )
```

La meta exige entregar ambos paquetes

No se prescribe dónde debe quedar el camión ni qué ruta utilizar.

El planificador puede escoger cualquier secuencia válida que satisfaga ambas condiciones.

```
1  (:goal (and
2    (at p1 city-d)
3    (at p2 city-d)
4  ))
```

El planificador encuentra una secuencia válida

Una solución de ocho acciones es:

```
move t1 city-b city-a
```

```
load p1 t1 city-a
```

```
move t1 city-a city-b
```

```
move t1 city-b city-c
```

```
load p2 t1 city-c
```

```
move t1 city-c city-d
```

```
unload p1 t1 city-d
```

```
unload p2 t1 city-d
```

Cada acción satisface sus precondiciones y el estado final cumple ambas condiciones de meta.

Resumen

- STRIPS representa un problema mediante estado inicial, meta y operadores.
- Las precondiciones determinan aplicabilidad; las listas add/delete producen el nuevo estado.
- PDDL separa las reglas reusables del dominio y los datos de cada problema.
- Un planificador puede buscar y validar planes sobre esta representación simbólica.



UNIVERSIDAD
SAN SEBASTIAN

USS-ICIFH001 • CLASE 08

¡Gracias

En el laboratorio escribirán y resolverán su propio problema PDDL.